Informalizing Lean

Aarne Ranta[1] and Claude[2]

[1] Department of Computer Science and Engineering, Chalmers University of Technology and University of Gothenburg, Sweden
aarne.ranta@cse.gu.se

[2] Claude Code, an AI coding agent by Anthropic, working in the first author's checkout

Abstract

Lean and its library Mathlib contain a large and growing part of mathematics in machine-checked form, but the formal code is hard to read for anyone who is not a Lean expert, and the code produced by autoformalization is read by even fewer. We describe a system that translates Lean declarations, with their statements, definitions and proofs, into ordinary mathematical language in several natural languages. The translation goes through the logical framework Dedukti and the Informath grammar, written in Grammatical Framework (GF), and is symbolic: it is compositional and does not invent content. The main bottleneck is the symbol table, which tells how each Lean constant is to be worded. We infer it for all of Mathlib's definitions from their docstrings and names, checking each candidate with the grammar and replacing the words that it lacks by words of its lexicon: 69% of 38,667 definitions get a wording, and Mathlib's own notation gives symbolic variants. Mathlib's classes, with their multiple inheritance, are read as a textbook defines algebraic structures, by extension of one parent ("a group is a monoid G with an operation $^{-1}$ on G such that ..."), and as predicates of their carriers. All definitions are then read in English, as formulas and in French, with their definientia: over 99% get a reading in English and in French, with the same symbol table, almost a third of them with no constant left symbolic. Theorems and proofs are read for 9,126 declarations of 82 modules, the undergraduate core of Mathlib from logic and orders to algebra, number theory, linear algebra, topology and analysis, in English, French and Czech: 72% are fully worded, and 7,594 of the 8,472 theorems get a proof text, by terms or by tactics. We compare the translation with lean2dk, which translates Lean to Dedukti for checking rather than for reading, and show how the comparison led us to keep implicit arguments and coercions in Dedukti and hide them only in the readings. The system was built in a collaboration between a human researcher and an AI coding agent, which is also a coauthor of this paper; we report what came from where.

1 Introduction: the documentation problem of formal code

Formal mathematics is precise but not readable. A theorem in Lean's Mathlib [29] is stated in a language designed for an elaborator, which resolves implicit arguments, type classes, coercions and notations; a proof is a term or a script of tactics whose meaning lies in the goals that only Lean displays. Mathlib has a documentation tradition that partly compensates for this: definitions must have docstrings, and doc-gen4 [28] renders the declarations with their types. But in our checkout of Mathlib, 83% of definitions, structures and classes have a docstring, and only 15% of theorems.

The problem becomes urgent with *autoformalization*. Large language models now produce formal mathematics at a scale that no human reviewer can follow [30]. A proof checker guarantees that the code is correct, but not that it formalizes what was meant: the theorem proved may be a different theorem. To check this, someone has to read the formal statements. An LLM can be asked to explain them, but then the explanation is no more reliable than the LLM [14, 10].

Symbolic informalization is the complement: a compiler-like translation from the formal language to natural language, which follows the formal structure and adds nothing that is not there [25]. Its traditional weaknesses are fluency, the effort of building it, and the restriction to one formal language [14]. The Informath project [26] addresses them with an interlingual architecture: Dedukti [3] as the hub of formal languages and the Grammatical Framework (GF) [21, 23] for natural languages.

This paper applies Informath to Lean and Mathlib. Our contributions are:

- an exporter from Lean 4 declarations, statements, definitions and proofs to Informath's Dedukti, and a pipeline that reads them in English, French, Czech, German and Swedish, as well as in LaTeX formulas, with term proofs and tactic proofs as texts (Section 5);
- a method for inferring the symbol table, the mapping of Lean constants to words, for all of Mathlib's definitions from their docstrings and names, with each candidate checked by the grammar, and with Mathlib's notation as symbolic variants (Section 7);
- a treatment of Mathlib's classes as algebraic structures defined by extension of a parent, and as defined predicates, with the GF judgements that read them (Section 8);
- a comparison with lean2dk, a translation of Lean to Dedukti for checking, and the consequences we drew from it: implicit arguments and coercions kept in Dedukti and hidden by the symbol table (Section 6);
- an account of how much comes from the existing grammar, from Mathlib's own text, from general lexical resources, from the human author and from the AI agent (Section 9);
- results on all of Mathlib's definitions and on the theorems of a part of Mathlib, examples good and bad, and the open problems (Sections 10–12).

The system is open source [27], with web pages showing all 38,667 definitions with their readings in English, as formulas and in French, the 9,126 statements and structures with their proofs in English, French and Czech, and a stand-alone program that informalizes any Lean file.

Authorship. The software described here and this paper were written by Claude, an AI model by Anthropic, working as a coding agent (Claude Code [2]) in the first author's checkout, in dialogue with him. The first author set the goals and directions, designed Informath before this project, made the decisions between alternatives, diagnosed some of the hardest problems, and reviewed the results; Claude wrote the code, ran the experiments and drafted the text. Section 9 makes the division more precise.

2 Lean and Mathlib: tool, resource, and test case

Lean 4 [8, 9] is a proof assistant and functional programming language based on dependent type theory. Mathlib, its community library, covers mathematics from algebra and number theory to analysis, topology and category theory; at the commit we used (`6972d52c0d`, 26 October 2026) it has about 8,660 files and 2.3 million lines, with about 194,000 theorems and 48,000 definitions, structures and classes.

Mathlib plays three roles for informalization. It is a *tool*: Lean's elaborator resolves what the source leaves implicit, and its environment gives each declaration its fully elaborated type, which is what we translate. It is a *resource*: its names follow strict conventions (`Nat.Prime.two_le`, `IsPrimePow`, `minFac`), and its docstrings are written in mathematical English ("`Nat.Prime p` means that `p` is a prime number"), so that both can be mined for wordings. And it is a *test*

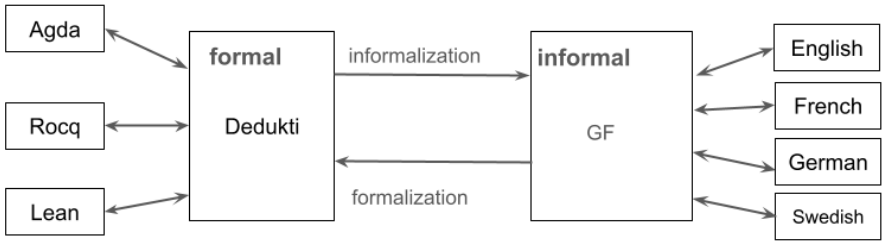


Figure 1: The architecture of Informath: Dedukti as the formal interlingua, GF as the informal one.

case: it is large and varied enough that no hand-written table can cover it, and its users are an audience who can judge the results.

3 GF, Dedukti, and Informath

Informath (Figure 1) has two interlinguas. On the formal side, **Dedukti** [3] is a minimal logical framework, the $\lambda\Pi$-calculus modulo rewriting, designed as a hub between proof systems. Informath uses a fixed Dedukti theory of sets, propositions and proofs:

```
Set : Type.   Elem : Set -> Type.   Prop : Type.   Proof : Prop -> Type.
and : Prop -> Prop -> Prop.   forall : (A : Set) -> (Elem A -> Prop) -> Prop.
```

Constants beyond these base constants are user constants, such as the ones of Mathlib.

On the informal side, **GF** [21, 23] is a grammar formalism with an *abstract syntax*, a system of typed trees, and a *concrete syntax* for each language, which maps the trees to strings and back. Informath's abstract syntax has categories such as `Prop`, `Kind`, `Exp`, `Proof`, and lexical categories such as `Noun`, `Adj`, `Fun` ("the X of Y"), `Verb2`; its concrete syntaxes are built with GF's Resource Grammar Library (RGL) [22], which implements inflection, agreement and word order for some 40 languages, so that a rule such as "x is A" is written once with `mkCl` and gets the right forms in each.

The mapping between the two is defined by a **symbol table** (`.dkgf`), which assigns to each Dedukti constant a wording, either a GF tree or an example string that the grammar parses:

```
Nat__minFac : "the smallest prime factor of #1" | $\operatorname{minFac}(#1)$
IsPrimePow  : "#1 is a prime power"
Eqt         : "#2 is equal to #3"     -- the type argument #1 is not shown
```

The string is parsed in a category `Example`, whose trees reveal the lexical category (`Fun`, `Noun1`, `Adj2`, ...) and the order of the arguments. From the table, a Dedukti statement is translated to a GF tree in a core language (MathCore), then rewritten by natural language generation (NLG) rules into variants (in situ quantification, aggregation of hypotheses, symbolic notation where it fits), which are ranked and linearized [25]. A constant without an entry is shown symbolically, as IsUnit(u).

4 Prior work

Readable formal mathematics has a long history. Mizar [4] was designed to look like mathematical text and produces the journal *Formalized Mathematics*. Coscoy et al. [5] extracted

texts from Coq (now Rocq) proof terms, which is close to what we do for Lean's term proofs. Isabelle/Isar [33] is a readable proof language; Verbose Lean [17] gives Lean a controlled natural language for teaching, in which tactic proofs read like texts, and inspired our treatment of tactic proofs. On the formal side, lean2dk [31, 32] translates Lean to Dedukti for checking; Section 6 compares it with our translation. In the other direction, controlled natural languages such as Naproche [7, 16], ForTheL [19] and MathNat [13] are parsed into formal languages. GF has been used for mathematical language from the start [11, 24, 15], and GFLean [20] autoformalizes a fragment of English into Lean with GF.

Neural approaches have used LLMs to informalize Lean, notably the Herald dataset of Mathlib statements annotated in natural language [10], and to autoformalize, with symbolically generated data as one source [14, 12]. Our work is complementary: the translation itself is symbolic, but the lexicon that it needs is inferred from Mathlib's own text, and the system was built with an AI agent.

5 From Lean code to natural language

The pipeline has four steps.

1. Export to Dedukti. A Lean program, `Export.lean`, imports Mathlib modules and translates each declaration's elaborated type, and the definiens of definitions and term proofs, to Informath's Dedukti. A binder $(x : \alpha)$ with α : `Type` becomes $(x$: `Elem` $\alpha)$, one with α : `Prop` a hypothesis, and $(\alpha$: `Type` $u)$ becomes $(\alpha$: `Set`$)$. Implicit arguments are kept, and the symbol table hides them in the readings (see below). An instance binder `[CommMonoid M]` becomes a hypothesis `Proof (CommMonoid M)`, a class is declared as a predicate (and defined as one, with its structure, in Section 8), and a bundled function applied by coercion (`DFunLike.coe f x`) becomes a constant `f__app` with its own type. A table of 39 mappings sends Lean's logic and arithmetic (`Dvd.dvd`, `Nat.Prime`, `HAdd.hAdd`, `Iff`) to Informath's base constants, whose wordings are part of Informath. The output is meant to be read, not checked: it has no universe levels, instance arguments or computation rules, which a type checker would need (Section 6). For example, the Lean theorem

```
theorem Irreducible.not_dvd_isUnit {M : Type*} [CommMonoid M] {p u : M}
    (hp : Irreducible p) (hu : IsUnit u) : ¬p ∣ u
```

is exported with its implicit variables and its instance as hypotheses, as a theorem is stated with all of them, and its proof term with them as λs:

```
def Irreducible__not_dvd_isUnit : (M : Set) -> (_ : Proof (CommMonoid M)) ->
    (p : Elem M) -> (u : Elem M) -> (hp : Proof (Irreducible p)) ->
    (hu : Proof (IsUnit u)) -> Proof (not (divide p u)) :=
  M => x => p => u => hp => hu =>
    mt (divide p u) (IsUnit p)
       (ifI (divide p u) (IsUnit p) (h => isUnit_of_dvd_unit M p u h hu))
       (Irreducible__not_isUnit M p hp).
```

Here `divide` and `not` are Informath's base constants, mapped from `Dvd.dvd` and `Not`, and `ifI` is the natural deduction rule of implication introduction. The formula reading of the statement is $M \in \mathrm{Set}, \mathrm{CommMonoid}(M), p, u \in M, \mathrm{Irreducible}(p), \mathrm{IsUnit}(u) \vdash \neg(p \mid u)$; its verbal reading is shown in Section 10.

Hidden arguments and coercions. The first version of the exporter dropped implicit arguments and erased coercions (casts between number types, the values of subtypes and of bundled sets), as a reader does not see them in Mathlib. Both were mistakes, which the comparison with lean2dk (Section 6) made clear. Dropping implicit arguments left the types of the constants open, as `HVAdd__hVAdd : (x1 : Elem X) -> (x2 : Elem Y) -> Elem Z` with free X, Y, Z, and made declarations and uses disagree: `Equivalence.mk`, a constant whose type is a proposition and is therefore stated with its implicit arguments, was applied without them, and the proof text read its arguments off the wrong positions, showing the proofs of reflexivity and symmetry as the relation. Erasing coercions turned 45 statements into tautologies: `AddSubgroup.coe_sub`, $\uparrow(x-y) = \uparrow x - \uparrow y$ (subtraction in a subgroup agrees with subtraction in the group), read "$x-y$ is equal to $x-y$". Now the exporter keeps both, and writes what the readings hide as lines of a generated symbol table:

```
HVAdd__hVAdd : (X : Set) -> (Y : Set) -> (Z : Set) -> Elem X -> Elem Y -> Elem Z.
#HIDE HVAdd__hVAdd 1 2 3         -- the three types are implicit
#COERCION Subtype__val 3         -- the coercion reads as its third argument
```

Informath removes the hidden arguments before it looks up the entries, so that the hand-written and generated entries count the explicit arguments as before, and a Mathlib update that moves an implicit argument changes only the generated lines. A coercion is read as its argument, unless that makes the two sides of an equation or an equivalence the same, and then it is shown: "$\uparrow(x_1 - y)$ is equal to $\uparrow x_1 - \uparrow y$". This leaves 11 tautological readings (of 45) and the number of fully worded readings as it was.

2. Readings. RunInformath translates the Dedukti declarations with the base table, Mathlib's table (Section 7) and the grammar, and gives a reading in each language and a formula in LaTeX. The index of all 9,126 statements is built in one call per mode, so that the grammar (8 MB) is loaded once; in English, French and Czech this takes about eleven minutes. German linearization turned out to be exponential in nested noun phrases (three levels of "the maximum of" took 35 seconds), so German is read apart, with a time limit of one second per statement, and it is not in the present index. A constant without an entry is shown as a formula, $\mathrm{c}(x, y)$, when its arguments are formulas, rather than as "c applied to x and y"; this took the phrase "applied to" from about 5,400 to about 1,500 occurrences in the readings and proof texts.

3. Term proofs. A definiens that is a proof term is translated to a sequence of proof units: each λ becomes an assumption ("Let n be an element of $\mathbb{N}$", "Assume that p is prime"), each application of a lemma a conclusion, justified by the lemma. Lean's logical constructors are mapped to natural deduction rules in Dedukti. The rewriting that `simp` and `rw` leave in a proof term (`congrArg`, `Eq.mpr`, `of_eq_true`) is one step, "By rewriting, ...", whose margin names the simp lemmas used; the text itself names no lemma, and the margin links each rule to its entry.

4. Tactic proofs. Most of Mathlib's proofs are tactic scripts, whose text says little: `rw [hn, mul_one]`. Following the idea of Verbose Lean [17], we use the goals instead. The exporter elaborates the module again with Lean's info trees, and records for each tactic of a sequence the goal before it and the goals after it, translated to Dedukti. A step then becomes units by comparing them: new hypotheses are assumed or stated, a closed goal is a conclusion, a changed goal is "it is enough to show that ...", several goals are cases. Each unit keeps the tactic as its justification:

```
theorem Prime.eq_one_or_self_of_dvd {p : ℕ} (pp : p.Prime) (m : ℕ) (hm : m ∣ p) :
    m = 1 ∨ m = p := by
  obtain ⟨n, hn⟩ := hm
  have := pp.isUnit_or_isUnit hn
  rw [Nat.isUnit_iff, Nat.isUnit_iff] at this
  apply Or.imp_right _ this
  rintro rfl
  rw [hn, mul_one]
```

1.	Let n be an element of $\mathbb{N}$ such that $p = m \times n$.	`obtain ⟨n, hn⟩ := hm`
2.	m is a unit or n is a unit.	`have := pp.isUnit_or_isUnit hn`
3.	$m = 1$ or $n = 1$.	`rw [Nat.isUnit_iff, Nat.isUnit_iff] at this`
4.	It is enough to show that if $n = 1$, then $m = p$.	`apply Or.imp_right _ this`
5.	Assume that $n = 1$.	`rintro rfl`
6.	$m = p$.	`rw [hn, mul_one]`

The same units are read in the other languages: "Stačí ukázat, že jestliže $n = 1$, pak $m = p$", "Il suffit de montrer que ...". A step that closes a goal or derives a fact says how, where its tactic tells: "By simplification" (`simp`, 1,405 lines), "By rewriting" (`rw`, 1,335), "By a routine argument" (`grind`, `aesop`, 290), "By definition" (`rfl`, 74), "By computation" (`ring`, `omega`, 33); a step that applies a lemma (`exact`, `apply`) is left as it is, as its tactic names the lemma. An additive theorem that `to_additive` makes from a multiplicative one has no source of its own, so it gets the proof of its original, with each goal translated by `to_additive`'s own table: 927 of the 1,116 such twins have a proof text.

Outputs. The results are shown on web pages, one per part of Mathlib, where each declaration has its Lean source, its Dedukti translation (folded), its readings and formula, and its proof text [27]. A stand-alone program, `mathlib-informath`, does the same for any Lean file or code on standard input, as plain text, HTML or PDF, in about 40 seconds, most of which Lean spends loading Mathlib.

For instance, the code on standard input

```
theorem t (n : ℕ) (h : 0 < n) : n ≠ 0 := by omega
```

gives, with `-languages=Eng,Ger`,

```
t (theorem)
  English: Let $n$ be a natural number such that $0 < n$. Then $n \neq 0$.
  German: Sei $n$ eine natürliche Zahl derart, dass $0 < n$. Dann $n \neq 0$.
  Proof, English:
    1. $n \neq 0$.   [omega]
```

A user's own symbol table can be added on top of Mathlib's, for the concepts that the code defines itself.

6 Reading versus checking: lean2dk

Our exporter is not the only translation of Lean to Dedukti. lean2dk [31], described in Vaishnav's PhD thesis [32], translates Lean so that Dedukti can *check* its proofs and pass them on to other systems. The two have different purposes, and almost every difference follows from that.

lean2dk reads Lean as a functional pure type system and uses the encoding of Cousineau and Dowek [6]: a type is a code `t : Univ l` decoded by `El l t`, and a dependent function

type is `Pi s1 s2 s3 A (x => B)`, with the universe of the function type the `imax` of the others. Universe levels, with Lean's universe polymorphism and its impredicative `Prop`, have an encoding of their own, a rewrite system that computes normal forms of level expressions. Lean's definitional equalities become rewrite rules: the unfolding of definitions, the reduction of recursors on constructors for every inductive type, projections, structure eta, `Quot.lift`. Three of them, proof irrelevance, unit eta and K-like reduction, cannot be rewrite rules, and are removed beforehand by Lean4Less, a modified Lean kernel that translates a Lean environment into a smaller theory without them, inserting casts where the original used them. Checking the result needs a patched Dedukti, and constants whose checking would need infeasible computation (large `Nat` literals in unary form, the exponential normal forms of `omega`'s machinery) are stubbed, that is, postulated.

Our exporter keeps none of this: types are `Set`, `Elem`, `Prop` and `Proof`, universes are collapsed, instance arguments are dropped, there are no recursors or rewrite rules, and constants are mapped to Informath's vocabulary where it has a word for them (`HAdd.hAdd` to `plus`). Only proofs written as terms in the source are translated, and tactic proofs are read from their goals. The difference shows on one theorem of Lean's library, `Int.ofNat_sub`: for natural numbers $m \leq n$, $\uparrow (n - m) = \uparrow n - \uparrow m$ in the integers. Our export is

```
Int__ofNat_sub : (m : Elem Nat) -> (n : Elem Nat) -> (h : Proof (Leq m n)) ->
  Proof (Eq (Nat__cast Int (minus n m)) (minus (Nat__cast Int n) (Nat__cast Int m)))
    .
```

and it has no proof, as the proof is a tactic. lean2dk's statement alone is about 1,300 characters, as it keeps `Nat.cast` with the instance `instNatCastInt`, `HSub.hSub` with its three types, levels and instance, and `El` and `Pi` at every binder; the proof is another 14,600 characters, and translating this one theorem brings in 430 constants, 700 KB of Dedukti in nine files, with 14 constants stubbed. The thesis reports output two to three orders of magnitude larger than Lean's own `.olean` files, and the translation of only lower-level parts of the standard library to the end [32, Section 7.1].

Comparing the two showed two flaws of ours, which Section 5 describes as fixed: before, our export of `Int.ofNat_sub` was `Eq (minus n m) (minus n m)`, a tautology, since the casts were erased; and our signatures had free type variables where implicit arguments had been dropped. lean2dk never identifies two things, so it cannot lose a statement this way. What we still leave out is what a checker needs and a reader does not: universes, instances and computation. The last of these has a cost for us too: most of the 42 proofs that get no text are ones where Lean accepts a λ as the proof of a proposition that is a $\forall$ only after a definition, such as `upperBounds`, is unfolded, which lean2dk's rewrite rules would do. The two translations are complementary: one gives a reading, the other a check, of the same declaration.

7 Inferring the symbol tables

Informath's own lexicon covers basic mathematics; Mathlib has tens of thousands of concepts. The first tables were written by hand: 108 entries for the constants that our first modules use, with suggestions from docstrings and names checked by the grammar. This does not scale, so we infer a table for all of Mathlib's definitions.

The definitions. `export -definitions` imports all of Mathlib (20 seconds) and exports every declaration that may need a wording: definitions, structures, classes, fields, inductive types, but not theorems, instances, tactics, notations, auxiliary or deprecated declarations.

Of 56,852 such constants, 38,667 remain after filtering, each with its Dedukti type, explicit argument names and docstring. Its type tells its category: a function or constant (`Exp`), a type (`Kind`), or a predicate (`Prop`).

Candidates. For each definition, candidate wordings are made in two ways. *From the docstring*, patterns pick noun phrases and predicates: "`‘f n‘` is the X of `‘n‘`", "The X of ... is", "A set is X if", with the backticked argument names replaced by their positions `#1`, `#2`. *From the name*, the last component is split at capitals and underscores, and Mathlib's abbreviations are spelled out (`minFac`: minimum factor; `card`: cardinality; `AddMonoidHom`: additive monoid homomorphism, with *Add* as a modifier rather than "sum"). The name gives "the X of `#1`" for a function, "X of `#1`" for a type, "`#1` is (a) X" for a predicate (`IsCountablyCompact`: "`#1` is countably compact"), "`#1` has X" for one named after a verb (`HasLimits`, `PreservesFiniteLimits`). A few conventions needed special rules: a constructor `Bicone.ofLimitCone` is named after its namespace ("the bicone of `#1`"), a function `pointOfClosedPoint` after its head, but a type `PartitionOfUnity` keeps its "of".

For example, `Ordinal.cof : Ordinal → Cardinal` has the docstring "The cofinality on an ordinal is the `Order.cof` of any isomorphic linear order". Its first pattern gives "the cofinality on an ordinal of `#1`", which does not parse; with the generic prepositional phrase cut out, "the cofinality of `#1`" does, and it is also what the name gives, as `cof` is spelled out as *cofinality*. Since the candidate shares a word with the name, it is accepted; a docstring wording that shares no word with the name, such as "the ring" for `HahnSeries.toPowerSeries`, is not tried.

Checking. All 41,000 candidates are parsed by the grammar in one call (30 seconds). A candidate is accepted if its parse has the category and the arity of the constant; the first one accepted becomes the entry. Many constants take explicit parameters before their main arguments (a category, a Grothendieck topology); a wording may leave them out, as `Eqt` above leaves out the type. But leaving out the wrong argument gives a false reading: `IsRelUpperSet s P` was first read "P is a relation upper set". A predicate of two objects is therefore read as binary, "`#1` is a relative upper set with respect to `#2`", with the subject chosen by the docstring (the argument that it does *not* name after "relative to", "for", "over"), and without such a clue it gets no reading. We prefer a symbolic reading to a wrong one.

The lexicon. Candidates fail mostly because of unknown words. These are added to the grammar: a word that recurs in the candidates gets a noun, adjective, adverb or verb entry, with its category and plural from the RGL's English dictionary (`DictEng`), and a word that the dictionary lacks is admitted if three docstrings use it. Junk had to be filtered out by hand-made lists: docstrings contain function words that the dictionary lacks (*because*, *without*) and Lean jargon (*simproc*, *defeq*). The result is 507 English words, from *cocone*, *equicontinuous* and *hypercover* to *Zariski*.

For the other languages, we first tried GF's WordNet [18, 1], choosing senses by their domain tags: only 117 of 789 words were found, 13 of them in a sense tagged *mathematics*, and even those were mixed (*hexagon*: *Sechseck*, correct; *minus*: *Weg*, wrong; *triplet*: *trois*, wrong). Mathlib's jargon is not there. We therefore keep the translations in a table, one line per word with its French, German, Swedish and Czech forms (gender, plural), written by Claude and to be reviewed, with an empty cell where unsure; the grammar modules are generated from it. Where a language lacks a word, the index keeps the reading made without the generated table, which shows the constant symbolically.

The lexicon, a second time. Most definitions left without an entry have a word in their name that the grammar lacks as a noun or adjective, although its lexicon (4,560 words) has a word for it. A second round replaces such words: a verb by its nominalization, after its object (`restrictScalars`: "the scalar restriction of #1"); an abbreviation by the word of the lexicon that it abbreviates and that the docstrings of the definitions named with it use most (`denom`: denominator, `coeq`: coequalizer); a proper name by its spelling (`kronecker`: Kronecker); and `desc` and `forget` by "the induced morphism" and "the forgetful functor". As long names read word by word were mostly wrong, the round is restricted to names of at most two words. It gives 887 entries with 135 replacements, which are listed for review.

Notation. The generated entries were verbal only, although Mathlib has notation for many of its concepts. The exporter therefore prints each definition applied to variables for its explicit arguments with Lean's own delaborator, with the scoped notations of its namespaces opened (`n !` is scoped in `Nat`). Where the result is not the name applied to the arguments, it is notation, and it is written in LaTeX token by token, the variables becoming the argument positions:

```
Int__floor : "the floor of #1" | $\lfloor #1 \rfloor$
RingEquiv  : "ring isomorphism of #1 and #2" | $#1 \simeq_{+*} #2$
NNReal     : $\mathbb{R}_{\geq 0}$       -- no wording: the notation alone
```

A run of symbols is one token, so that Lean's `M ≃+* N` and $\mathbb{R}\geq 0$ get their own LaTeX, and a token without one rules the notation out, as do binders ($\sum$ `x ∈ s, f x`) and Lean's bracketed subscripts (`M ⊗[R] N`, `f^[n]`), which LaTeX writes otherwise. This first gave 42 definitions a symbolic variant, among them $\bot$, $\top$, s^c, $|x|$, $\|x\|$, $\lceil x \rceil$, $s \,\Delta\, t$, $\sqrt{x}$, A^T and $\#s$. Logging the tokens that had no LaTeX showed what was missing: with some sixty more (Lean's $+_v$, $\to_*$, $^\times$, $\bigcup_0$, Greek letters), 115 definitions have one, among them $x +_v s$, $M^\times$, $M \to_* N$, $R \to_{+*} S$, $\bigcup S$ and π. The variant matters beyond the definitions: an equation is written as a formula only if all its constants have a symbolic form, so that the cosets $x +_v s$ of a subgroup were read "the sum of x and s" before. The number is still small because the most common notations ($+$, $|$, $\leq$, $n!$) belong to constants that are mapped to Informath's own, and many others are binders. (The definition pages of Section 11 were read with the 42.)

8 Structures and inheritance

Much of Mathlib's mathematics lives in its *classes*: `Monoid`, `Group`, `Ring`, `Field`, `Lattice`, `Module`, `TopologicalSpace`, some 2,000 in all. A class is a structure, a record of data and laws over a carrier type, and it is *extended* by other classes, often by several at once:

```
/-- A ‘Monoid‘ is a ‘Semigroup‘ with an element ‘1‘ such that ‘1 * a = a * 1 = a‘.
    -/
class Monoid (M : Type*) extends One M, Mul M, Semigroup M, MulOneClass M, NPow M
    where
  npow := npowRecAuto
  protected npow_zero (x : M) : x ^ 0 = 1 := by intros; rfl
  protected npow_succ (n : ℕ) (x : M) : x ^ (n + 1) = x ^ n * x := by intros; rfl

class Group (G : Type*) extends DivInvMonoid G where
  protected inv_mul_cancel : ∀ a : G, a⁻¹ * a = 1
```

Statements use the classes as predicates of their carriers: `[Group G]` becomes "G is a group". But what a group *is* is spread over the hierarchy, and Lean's way of putting it together is not a

textbook's. Lean flattens: an instance of `Monoid` has all the fields of all its ancestors, and the ancestors include classes that exist only to carry notation (`One`, `Mul`), technical classes that fix the implementation (`NPow`, the power x^n with a default definition, and `DivInvMonoid`, a monoid with an inverse and a division but no laws), and the additive copy of every multiplicative class, generated by the attribute `to_additive`. A textbook names one parent and lists what is new: *a monoid is a semigroup with an identity element; a group is a monoid in which every element has an inverse; a commutative group is a group in which* $ab = ba$.

Four options. The design started from a note of options, written by Claude at the first author's request, each option with samples read by Informath. (A) *Flattened*: each class a structure with all its fields; complete and correct, but a commutative group repeats the monoid and the group. (B) *The parent as a clause*: the structure has its carrier, its parent's predicate as a hypothesis, as Lean's instance argument, and only its own fields. (C) *An extension judgement*: B's encoding, read by a new GF judgement as a definition by extension. (D) *The class as a defined predicate*: "G is a group if G is a monoid and there is a function i such that ...", its data quantified existentially. D unifies the class with the predicate that statements use, and fits proofs: a use of the field `mul_comm` becomes an elimination of the definition, "since G is a commutative group, $ab = ba$"; it loses that the identity and the inverse are data chosen by the structure. We implemented B and C for the structure and D for the predicate, and show both.

The export. For a class C with carrier G and primary parent P, the exporter writes a structure and a definition:

```
Monoid__structure : (M : Set) -> (h : Proof (Semigroup M)) -> ({|1|} : Elem M) ->
  (one_mul : Proof (forall M (a => Eqt M (times {|1|} a) a))) ->
  (mul_one : Proof (forall M (a => Eqt M (times a {|1|}) a))) -> Structure.
def Monoid : (M : Set) -> Prop := M => and (Semigroup M)
  (exists M (e => and (forall M (a => Eqt M (times e a) a)) (forall M (a => Eqt M (
    times a e) a)))).
```

Several choices make this read like mathematics:

- *The primary parent* is the one the docstring names, as Mathlib writes its docstrings as a textbook would ("A `Monoid` is a `Semigroup` with ...", "A `Group` is a `Monoid` with an operation $^{-1}$ ..."), skipping `MulOneClass` and `DivInvMonoid`; without one, the ancestor with the most fields. The fields of the parent are its own: the structure lists only the fields that are new with respect to it.
- *Fields are named by their notation*: the identity is a variable 1 (written `{|1|}` in Dedukti), the inverse $^{-1}$, the order $\leq$. A field has a type by its shape: a binary operation `Oper G`, a unary one `UOper G`, written a^{-1} or $-a$, a relation `Rel G`, an action `Action R M` of a scalar on the carrier. The parent's operations are written as statements write them ($a \times b$, $a \leq b$).
- *Technical fields are left out*: a field with a default value (`npow`, `zpow`, `div`), data not built of the carrier (`natCast`, `nnqsmul`), and the laws that mention them. They are listed on the page.
- *Parameters*: a class over another type, as `Module R M`, has its scalars and their instance (`[Semiring R]`) as clauses before the carrier, which are read as a hypothesis. Greek carriers (α) get a textbook letter (R for a ring, K for a field, M for a monoid or a module), and out-parameters (α, β of a class of morphisms `F`) are parameters, `F` being the carrier.

- *Additive classes* are derived from the multiplicative ones by `to_additive`'s own translation table, which maps the names of the class, its parent and its fields, and the notation ($*$ to $+$, 1 to 0, a^{-1} to $-a$).

The grammar. Informath had a `StructureJmt`, which reads a structure as a list of clauses ("a monoid is an algebraic structure, where M is a set, M is a semigroup, 1 is an element of M, ..."). We added judgements of definition by extension to its abstract syntax:

```
ExtensionJmt      : Label -> Kind -> Kind -> Ident -> [ArgKind] -> Prop -> Jmt
ExtensionHyposJmt : Label -> [Hypo] -> Kind -> Kind -> Ident -> [ArgKind] -> Prop ->
    Jmt
```

with the defined kind, the parent kind, the carrier, the new data and their laws (and variants without data or without laws). They are produced by an NLG rule from a `StructureJmt` whose carrier is followed by its parent read as a noun (from the parent's entry, "`#1` is a semigroup"); the clauses before the carrier become hypotheses. The concrete syntax is written once with the RGL, as "a K is a P x with D such that Q", and the semantics maps the judgements back to `StructureJmt`, so that the round trip to Dedukti is preserved. Aggregation of quantifiers inside conjunctions ("for all elements a and b of G", "for all a, P and Q") makes the laws shorter. The structures of rings, orders, lattices and modules then read:

> A group is a monoid G with an operation $^{-1}$ on G such that for all elements a of G, $a^{-1} \times a = 1$.
> A ring is a semiring R with an operation $-$ on R such that for all elements a of R, $-a + a = 0$.
> A distributive lattice is a lattice G such that for all elements x, y and z of G, $(x \sqcup y) \sqcap (x \sqcup z) \leq x \sqcup (y \sqcap z)$.
> Let R be a semiring. Then a module over R is an additive commutative monoid M with an action $\bullet$ of R on M such that for all elements x and y of R, for all elements b of M, $(x \times y) \bullet b = x \bullet (y \bullet b)$, ...
> Un groupe est un monoïde G avec une opération $^{-1}$ sur G tel que pour tous les éléments a de G, $a^{-1} \times a = 1$.

and the predicates:

> Let M be a set. Then M is a monoid, if M is a semigroup and there exists an element e of M, such that for all elements a of M, $e \times a = a$ and $a \times e = a$.

Two pages of the index show 50 structures and predicates, from magmas to fields, from preorders to distributive lattices, and modules, in all five languages; 50 of 50 are fully worded in English.

All the classes. The same export runs over all of Mathlib's classes, with all classes as possible parents, for the definition pages (Section 11). 1,889 of the 2,009 classes export as structures. On the pages, a class worded as a noun is read as its structure, with its predicate under it; any other class as its predicate ("G is unique, if Inhabited holds for G and for all elements a of G, $a =$ Inhabited__default"); a class whose fields are all technical, such as `TopologicalSpace` with its `IsOpen`, keeps its declaration. Of the 1,948 classes on the pages, 359 read as extensions ("a metric space is a pseudo metric space G such that for all elements x and y of G, if the distance of x and y is equal to 0, then $x = y$"), 510 as defined predicates, and 86 as lists of clauses, as their parent is an adjective (`IsDomain`: "G is cancellative") or has two arguments (`Module R M` by `DistribMulAction R M`). 877 have no fields of their own: they combine parents (`LeftCancelSemigroup extends Semigroup, IsLeftCancelMul`), and the parents other than the primary one are not yet clauses, so these keep the plain reading.

	functions, constants	types	predicates	all
definitions	31,858	2,464	4,345	38,667
with a wording	23,059	1,816	1,866	**26,741** (69%)
read in English (Section 11): with a reading				**38,517** (99.6%)
with no constant shown symbolically				about 12,000 (31%)
read as a formula				**38,546** (99.7%)
read in French: with a reading				**38,306** (99.1%)
with no constant shown symbolically				about 11,950 (31%)
the wordings from: names 22,304; docstrings 3,488; names with the lexicon 887; the hand-written table 62				
symbolic variants from Mathlib's notation: 42; classes read as structures or predicates: 869				

Table 1: The symbol table of Mathlib's definitions.

Already in Informath	**From the first author**	**By Claude**
The GF grammar and its concrete syntaxes on the RGL; the Dedukti theory; symbol tables with verbal, symbolic and macro forms; NLG with variants and ranking; proof texts from terms	The goal; the choice of Mathlib's prime numbers as the first case; the contents of the pages; loading the grammar once; the idea of tactic proofs as proof units; the time limit for German; WordNet as a resource; a reviewable table rather than generated code; the stand-alone program; structures as in textbooks, over the hierarchy and then all classes; symbolic variants; the six tiers of modules with their proofs, English first; French and Czech for them; the verbose proofs to be explained; `-force-symbolic` for unworded constants; the comparison with lean2dk; a coercion operator hidden by the table, and hiding by the table rather than by the exporter	`Export.lean` with its translation rules; the index and the pages; the table inference, the lexicon generation and the second round with the lexicon; the translations table; the Informath extensions this needed (multilingual proof texts, `-tactic-proof`, aggregated hypotheses, Czech and Swedish modules, the extension judgements); the choice among the structure options, the parent from the docstring; notation from the delaborator; the proof rounds (rewriting as one step, the methods of tactic steps, the proofs of additive twins); the diagnosis of the implicit-argument mismatch; formulas for unworded constants; the Czech proof units and grammar fixes; the lean2dk analysis; `#HIDE` and `#COERCION` with the tautology guard

Table 2: The division of labour.

9 Who contributed what

Table 1 shows where the wordings of the definitions come from: most from names, an eighth from docstrings, 887 from names with words of the lexicon, and a few from the hand-written table, which overrides the generated one. Table 2 divides the work between the existing Informath, the first author's prompts and decisions, and the AI agent.

For the structures, the first author set the goal, informal definitions by extension rather than the flat records of the first version, and the scope, first the hierarchy from magmas to fields, lattices and modules, then all the classes on the definition pages; Claude wrote the design note with its options and samples, including the suggestion to take the parent from the docstring, chose among the options (B and C with D), and wrote the grammar judgements. Some of the decisive steps were the human's: the diagnosis of a parameter explosion in the GF compiler (a list category whose concrete functions grew to 5×10^{13}), the reason why size-optimized

grammars broke all proof texts (the proof units were not reachable from the start category), and the decision to keep the translations reviewable. Others were the agent's: the iteration over hundreds of failing cases, measuring each change on the whole of Mathlib, and testing a resource (WordNet) before relying on it. The last rounds show the pattern on a small scale. The first author noticed that a proof read "`forallI` applied to ..." and "the sum of x_5 and s"; Claude traced the first to the implicit arguments of `Equivalence.mk` and the second to a missing notation token. The first author suggested `-force-symbolic` for the remaining "applied to"; Claude found that it missed the main case, constants without entries, and made those formulas instead. Asked to compare the exporter with lean2dk, Claude found the tautologies of erased coercions; the first author proposed a coercion operator that the symbol table hides, and asked whether entries could hide implicit arguments too, which led to the generated `#HIDE` lines. All the words written by Claude, in all languages, are to be reviewed by native speakers.

10 Examples

Definitions. Mathlib defines a prime number as an irreducible element:

```
def Prime (p : ℕ) := Irreducible p
```

> Let $p \in \mathbb{N}$. Then p is prime, if p is irreducible.
> Soit $p \in \mathbb{N}$. Alors p est premier si p est irréductible.
> Låt $p \in \mathbb{N}$. Då är p primt, om p är irreducibelt.

Informath prefers the symbolic declaration of a variable of a set, "Let $p \in \mathbb{N}$", to the verbal "Let p be a natural number", also with a condition: "Let $p, u \in M$ such that ...". Wordings from the generated table, with the arguments shown by their names: "the cofinality of o", "the normalizer of H", "C has limits", "F preserves finite limits", "P is separated with respect to J", "M is a perfect matching".

Theorems. Euclid's theorem in five languages:

```
theorem exists_infinite_primes (n : ℕ) : ∃ p, n ≤ p ∧ Prime p
```

> Let $n \in \mathbb{N}$. Then there exists a natural number p, such that $n \leq p$ and p is prime.
> Soit $n \in \mathbb{N}$. Alors il existe un nombre naturel p tel que $n \leq p$ et p soit premier.
> Sei $n \in \mathbb{N}$. Dann gibt es eine natürliche Zahl p, derart, dass $n \leq p$ und p prim ist.
> Låt $n \in \mathbb{N}$. Då finns det ett naturligt tal p, så att $n \leq p$ och p är primt.
> Nechť $n \in \mathbb{N}$. Tedy existuje přirozené číslo p, takový, že $n \leq p$ a p je prvočíselné.

The aggregation of hypotheses makes statements with many variables readable:

> Let M be a set such that M is a commutative monoid. Let $p, u \in M$ such that p is irreducible and u is a unit. Then p does not divide u.

(`Irreducible.not_dvd_isUnit`; the Lean statement has the implicit `M`, `p`, `u` and the instance `[CommMonoid M]`.)

Term proofs. The definiens of `exists_infinite_primes`, with its local definitions and `have`s, becomes a proof text of twelve lines, which starts:

> 1. Let n be an element of $\mathbb{N}$. 2. $0 < n!$. 3. The successor of 0 is less than the successor of $n!$. 4. $n! + 1 \neq 1$. 5. minFac$(n! + 1)$ is prime. ...

Each line is justified by the lemma it applies (`Nat.factorial_pos`, `Nat.succ_lt_succ`, ...), linked to its own entry. The proof that the relation of having the same left coset, $x +_v s = y +_v s$, is an equivalence,

```
@Equivalence.mk _ (LeftCosetEquivalence s) (fun _ => rfl) Eq.symm Eq.trans
```

becomes sixteen lines, in three parts for reflexivity, symmetry and transitivity, of which the first is

> 1. To prove that for all elements x_1 of X, $x_1 +_v s = x_1 +_v s$, let $x_1 \in X$. 2. $x_1 +_v s = x_1 +_v s$. `[rfl]` 3. For all elements x_1 of X, $x_1 +_v s = x_1 +_v s$. `[forallI 1, 2]`

and the last "LeftAddCosetEquivalence(s) is an equivalence relation" `[Equivalence.mk 3, 8, 15]`, in Czech "je relace ekvivalence".

Bad cases. Not everything reads well:

- a constant without a wording stays symbolic: "Let n be a natural number such that Coprime$(2, n)$" in French, where the English has "2 and n are coprime";
- mismatched agreement: Czech *takový* above, which should be *takové*; "Let s be a upper set";
- literal wordings: a table entry given as a string is parsed word by word, so that "is equal to" became Czech "je rovná do" (*rovná* takes the dative) until the entry was written as the grammar's lexical `equalset_Adj2`;
- generated wordings that say little: `BoundedOrderHom.dual`, a bundled map without explicit arguments, is "the dual";
- German: "Dann $2 \leq p$" without a verb, and 51 statements over the time limit.

11 Results

Table 3 shows the readings of 9,126 statements from 82 modules. The first 755, from 23 modules, are Mathlib's order and lattice definitions, the algebra that the prime numbers import, the algebraic and order structures of Section 8, the prime numbers and prime factorizations. The other 8,371 are the undergraduate core that much of Mathlib imports, in six tiers: (A) logic, sets and orders, (B) groups and rings, (C) elementary number theory, (D) finite sets and sums, (E) linear algebra, ideals and polynomials, (F) topology and analysis. A statement is *fully worded* if no Lean constant or other name is shown as c. Every statement has a formula. The French readings are fully worded where the English ones are, as they use the same trees; Czech falls short by 87, mostly in group theory, where some words have no Czech form yet. In 144 statements a language keeps an older, more symbolic reading, made without the generated table, because the table's reading needs a word that the language lacks.

	statements	English	French	Czech
Order, algebra, structures, primes	755	641	641	641
A. Logic, sets, orders	1,940	1,553	1,553	1,554
B. Groups and rings	2,276	1,618	1,618	1,548
C. Elementary number theory	892	718	718	706
D. Finite sets and sums	767	547	547	544
E. Linear algebra, ideals, polynomials	1,091	666	666	664
F. Topology and analysis	1,405	862	862	861
all	9,126	**6,605**	**6,605**	**6,518**

Table 3: Fully worded statements, by language.

	examples	
good	"C has injective resolutions", "f is differentiable", "the wide pullback of s", "the Grothendieck topology of T", "the smallest prime factor of x"	22
acceptable	"the evaluation of g and p" (`MvPolynomial.eval₂`), "the adjunction", "the tail of p and *len_pos*" (a proof argument shown)	20
wrong	"the vector multiplicative vector of w and v" (`vecMulVec`), "the unit product of u and w" (`unitsSMul`), "the opposite opposite of S" (`opUnop`), "R is a continuous square root" (a class saying that one exists)	18

Table 4: A random sample of 60 generated wordings, judged by the second author.

Proofs. Of the 8,472 theorems, 7,594 (90%) have a proof text in each of the three languages: 3,653 from proof terms and 3,941 from tactic proofs, in 26,350 lines, of which 166 (0.6%) say no more than "it is obvious", for a step whose goal could not be read. Of the 878 theorems without a text, 42 fail in Informath, mostly where a λ proves a proposition that is a $\forall$ only after a definition is unfolded (Section 6); the others have proofs that the exporter does not translate, such as compiled pattern matching or terms too long to read. The export of the 82 modules, with their tactic proofs, takes about ten minutes, and the index about eleven.

For all of Mathlib, the generated table gives 69% of the definitions a wording (Table 1), checked by the grammar for category and arity but not reviewed by hand. Random samples during the development showed classes of errors, which led to the rules of Section 7.

A sample. To estimate the quality of the final table, Claude judged a random sample of 60 generated wordings against the definitions' docstrings (Table 4): a wording is *good* if it says what the definition is in idiomatic terms, *acceptable* if it is vague but not wrong, and *wrong* if it says something else or nothing. This is the judgement of one annotator, the second author, and is not yet reviewed.

Two thirds are thus usable, and the predicates fare best: of the eleven in the sample, nine are good. The wrong ones are mostly composite names read word by word (`presheafHom-SectionsEquiv`: "the presheaf homomorphism section equivalence"), and abbreviations spelled out wrongly in context (*SMul* is a scalar multiplication, not a product). Proof arguments and unnamed arguments (#4) should not be shown, and `LibraryNote`, documentation without mathematical content, should have been filtered out.

A sample of the second round. 60 of the 887 entries made with the lexicon, judged in the same way, are 40 good, 14 acceptable and 6 wrong (10%, against 30%): "the scalar restriction of f" and "f is differentiable on s" are good; "the composition application of c and i" (`applyComposition`) and "the finite extension of K and L" (`finiteExts`, a set of extensions) are wrong.

All the definitions in English. Finally, all 38,667 definitions were read in English and as formulas, and then in French, as the statements on the pages are: each definition's statement, with its definiens where it has one (25,491 of the exported constants have one that the exporter translates), and each class as its structure or predicate (Section 8), by the full symbol table. With eight RunInformath processes and a time limit of one second per definition and mode, this took about an hour on a laptop, and those over the limit were read again on a quiet machine. 38,517 definitions (99.6%) got a reading, 119 went over the time limit, and 31 got none; 38,546 got a formula. About 12,000 readings (31%) show no constant symbolically; the others show the constants that have no wording, often from the definiens, as operators. (The share was 33% before the classes were read as structures: a class's fields bring in more constants than its bare declaration, "we can say that M is a monoid".) The readings are published on 111 static pages of at most 400 definitions, by part of Mathlib, with a search page over all of them. For example, `Quiver.IsThin`:

> Let V be a set. Then V is thin, if for all elements a and b of V, the set of homomorphisms of a and b is a subsingleton.
>
> $V \in \mathrm{Set} \vdash \mathrm{IsThin}(V) :\Leftrightarrow \forall a \in V.\ \forall b \in V.\ \mathrm{Subsingleton}(\mathrm{Hom}(a, b))$

French uses the same pipeline and symbol table, as Informath parses the English wordings into GF trees and linearizes them in French; the French rows are hidden on the pages until asked for, as on the other pages. The French reading of `Quiver.IsThin` is

> Soit V un ensemble. Alors V est fin si pour tous les éléments a et b de V, l'ensemble d'homomorphismes de a et b est un sous-singleton.

where *fin* for *thin* is a word to be reviewed. In the first run, 9,774 of the French readings (26%) used a word that had no French form: 263 words of Informath's lexicon, from its parts that were written for English only (the Godement, HoTT and Naproche lexicons), led by *opposite* (in 1,554 readings), *lift* (1,053) and *carrier* (950). Claude wrote their French forms (*opposé*, *relèvement*, *ensemble sous-jacent*), to be reviewed, and the affected definitions were read again. Now 38,306 definitions (99%) have a French reading, and about 11,950 of them show no constant symbolically, almost as many as in English (about 12,000). 48 readings still use a word without a French form, which the pages show in English, marked. The other three languages can be read in the same way.

12 Known problems and open issues

Coverage. All of Mathlib's definitions are read, but only in English, as formulas and in French, and its theorems only in part: the 9,126 statements of the pages are a small part of its 194,000 theorems, which the same pipeline could read, with their proofs, but which we have not run on; German and Swedish are not in the present index. Of the definitions, 119 go over the time limit in English and 243 in French, and two thirds of the readings still show some constant symbolically, mostly constants of the definientia that have no wording. The generated table is

unreviewed, and in our samples 18 of 60 of its first wordings (30%) and 6 of 60 of those made with the lexicon (10%) are wrong: composite names (`quotientIdealMapEquivTensorQuot`) become nonsense when read word by word; bundled maps without arguments get bare readings; about 2,500 unknown words are abbreviations of jargon that we do not spell out. Long `simp` goals make poor reading, and limits and filters are read only as formulas ("Tendsto(f, l_1, l_2)" for "f tends to ...").

Structures. Multiple inheritance is reduced to one parent: the other parents of a class (`IsLeftCancelMul` of `LeftCancelSemigroup`) should be clauses, "a left cancellative semigroup is a semigroup that is left cancellative"; 877 classes wait for this. A parent that is an adjective ("cancellative") or that has two arguments (`DistribMulAction R M`) gives a list of clauses rather than an extension. D', the informal form of D ("for every a there is a b such that $ba = 1$" rather than "there is a function i"), is not generated, and the parent's multiplication is written $a \times b$ where a textbook writes ab.

Language. Agreement of "such that" with an existential's kind fails in Czech and French; multi-letter variables (`self`, `motive`) are shown as constants; a lemma used unapplied in a proof term gives garbled text, as does a coercion used unapplied (↑ alone); a binder over a proof ("for all elements h of Proof applied to ...") should read "if ...". About 1,500 occurrences of "applied to" remain where an argument is itself in words, and 11 readings are still tautologies. The French and Czech words added for the definitions and the theorems, and the readings themselves, are not checked. "An unary operation": the RGL's English article takes *an* before every word beginning with *un*. The words of four languages need native review, and German linearization needs a fix in the RGL.

Correctness. The translation is compositional, and the exporter now keeps implicit arguments and coercions, so that its declarations are closed and its applications match them. But it leaves out universes, instances and computation rules, so the Dedukti code is not type-checked (Section 6); a reading is as correct as the table entries it uses, and the generated entries are checked only by the grammar. A check of the exported Dedukti against its own signatures would catch errors like the shifted arguments of `Equivalence.mk` automatically, and lean2dk's check of the same declarations would guarantee what the readings say; a back-translation from the readings to Dedukti would close the loop, as Informath supports in the other direction.

13 Conclusion

Symbolic informalization of Lean is feasible at the scale of Mathlib if the lexicon can be inferred: Mathlib's naming conventions and docstrings, with the grammar's own lexicon, give wordings to 69% of its definitions, and a grammar can check them. With them, all of Mathlib's definitions have been read in English, as formulas and in French, almost a third of them with no symbolic residue in English, in an hour on a laptop; and 9,126 statements of its undergraduate core, with 7,594 proofs, in English, French and Czech, 72% of them fully worded. A translation for reading is not a translation for checking: lean2dk keeps everything a checker needs and is unreadable, ours keeps what a reader needs, and the comparison showed where we had kept too little, the coercions and implicit arguments that a reader does not see but a statement may depend on. Mathlib's own structure helps too: its docstrings name the parent a textbook would, its `to_additive` table gives the additive structures, and its delaborator the notation,

so that its classes read as definitions by extension. The readings are faithful by construction, and they come in five languages at once. What the method does not give is judgement: which wording is right, which word a language uses, when a reading is false although it parses. That remained the human's part, in a collaboration where the AI agent did most of the construction and the measuring.

Acknowledgements. The first author thanks Krasimir Angelov for the RGL work on Czech and German that this project relied on, and Josef Urban for introducing him to agentive coding. This work has been supported by the European Research Council through the grants *NextReason* (Grant ID 101200949) and *MALINCA* (Grant ID 101167526).

References

[1] Krasimir Angelov. A parallel WordNet for English, Swedish and Bulgarian. In *Proceedings of the 12th Language Resources and Evaluation Conference (LREC 2020)*, pages 3008–3015, 2020.

[2] Anthropic. Claude Code. https://claude.com/claude-code, 2025.

[3] Ali Assaf, Guillaume Burel, Raphaël Cauderlier, David Delahaye, Gilles Dowek, Catherine Dubois, Frédéric Gilbert, Pierre Halmagrand, Olivier Hermant, and Ronan Saillard. Dedukti: a logical framework based on the $\lambda\pi$-calculus modulo theory, 2023.

[4] Grzegorz Bancerek, Czesław Byliński, Adam Grabowski, Artur Korniłowicz, Roman Matuszewski, Adam Naumowicz, Karol Pak, and Josef Urban. Mizar: State-of-the-art and beyond. In Manfred Kerber, Jacques Carette, Cezary Kaliszyk, Florian Rabe, and Volker Sorge, editors, *Intelligent Computer Mathematics*, pages 261–279, Cham, 2015. Springer International Publishing.

[5] Yann Coscoy, Gilles Kahn, and Laurent Thery. Extracting text from proofs. In Mariangiola Dezani-Ciancaglini and Gordon Plotkin, editors, *Proc. Second Int. Conf. on Typed Lambda Calculi and Applications*, volume 902 of *LNCS*, pages 109–123, 1995.

[6] Denis Cousineau and Gilles Dowek. Embedding pure type systems in the lambda-pi-calculus modulo. In *Typed Lambda Calculi and Applications (TLCA 2007)*, volume 4583 of *LNCS*, pages 102–117. Springer, 2007.

[7] Marcos Cramer, Bernhard Fisseni, Peter Koepke, Daniel Kühlwein, Bernhard Schröder, and Jip Veldman. The Naproche Project Controlled Natural Language Proof Checking of Mathematical Texts. In Norbert E. Fuchs, editor, *CNL*, volume 5972 of *LNCS*, pages 170–186. Springer, 2009.

[8] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. The Lean theorem prover (system description). In Amy P. Felty and Aart Middeldorp, editors, *Automated Deduction - CADE-25*, pages 378–388, Cham, 2015. Springer International Publishing.

[9] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *LNCS*, pages 625–635. Springer, 2021.

[10] Guoxiong Gao, Yutong Wang, Jiedong Jiang, Qi Gao, Zihan Qin, Tianyi Xu, and Bin Dong. Herald: A natural language annotated Lean 4 dataset. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025.

[11] Thomas Hallgren and Aarne Ranta. An Extensible Proof Text Editor. In Michel Parigot and Andrei Voronkov, editors, *LPAR-2000*, volume 1955 of *LNCS/LNAI*, pages 70–84. Springer, 2000. http://www.cse.chalmers.se/~aarne/articles/lpar2000.pdf.

[12] Pei Huang, Nicholas Smallbone, and Aarne Ranta. Multilingual Autoformalization via Fine-tuning Large Language Models with Symbolically Generated Data. *RISC Proceedings on Symbolic Computation and Machine Learning*, 1:3–20, 2025.

[13] Muhammad Humayoun and Christophe Raffalli. MathNat - Mathematical Text in a Controlled Natural Language. *Journal on Research in Computing Science*, 66, 2010.

[14] Albert Q. Jiang, Wenda Li, and Mateja Jamnik. Multi-language diversity benefits autoformalization. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 83600–83626. Curran Associates, Inc., 2024.

[15] Josefin Kelber, Michael Kohlhase, Jan Frederik Schaefer, and Marcel Schütz. Reaping the benefits of modularization in flexiformal mathematics by GF-based AST transformations. In *Intelligent Computer Mathematics: 18th International Conference, CICM 2025, Brasilia, Brazil, October 6–10, 2025, Proceedings*, page 191–207, Berlin, Heidelberg, 2025. Springer-Verlag.

[16] Peter Koepke. A Natural Language Formalization of Perfectoid Rings in Naproche. In Yannick Forster and Chantal Keller, editors, *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 6:1–6:15, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[17] Patrick Massot. Teaching Mathematics Using Lean and Controlled Natural Language. In Yves Bertot, Temur Kutsia, and Michael Norrish, editors, *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 27:1–27:19, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[18] George A. Miller. WordNet: A Lexical Database for English. *Commun. ACM*, 38(11), 1995.

[19] Andrei Paskevich. The Syntax and Semantics of The ForTheL Language, 2007.

[20] Shashank Pathak. GFLean: An Autoformalisation Framework for Lean via GF. arXiv 2404.01234, 2024.

[21] Aarne Ranta. Grammatical Framework: A Type-Theoretical Grammar Formalism. *The Journal of Functional Programming*, 14(2):145–189, 2004.

[22] Aarne Ranta. The GF Resource Grammar Library. *Linguistics in Language Technology*, 2, 2009.

[23] Aarne Ranta. *Grammatical Framework: Programming with Multilingual Grammars*. CSLI Publications, Stanford, 2011.

[24] Aarne Ranta. Translating between language and logic: What is easy and what is difficult. In Nikolaj Bjørner and Viorica Sofronie-Stokkermans, editors, *Automated Deduction – CADE-23*, pages 5–25, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.

[25] Aarne Ranta. Towards multilingual autoformalization and informalization of mathematics. In *SLTC-2024, Swedish Language Technology Conference*, 2024. https://sltc2024.github.io/abstracts/ranta.pdf.

[26] Aarne Ranta. Informath: Informalization and Autoformalization of Formal Mathematics. https://github.com/GrammaticalFramework/informath, 2026.

[27] Aarne Ranta and Claude. mathlib-informath: Informalizations of Lean Mathlib4. https://github.com/aarneranta/mathlib-informath, 2026.

[28] The Lean community. doc-gen4: Document generator for Lean 4. https://github.com/leanprover/doc-gen4, 2025.

[29] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020.

[30] Josef Urban. 130k lines of formal topology in two weeks: Simple and cheap autoformalization for everyone? arXiv 2601.03298, 2026.

[31] Rishikesh Vaishnav. lean2dk: A translator from Lean to Dedukti. https://github.com/Deducteam/lean2dk, 2026.

[32] Rishikesh Vaishnav. *Translating Proofs from Lean to Dedukti*. PhD thesis, Université Paris-Saclay, École doctorale STIC, Laboratoire Méthodes Formelles, 2026. Supervised by Frédéric Blanqui; defended at ENS Paris-Saclay on 10 March 2026; NNT 2026UPASG009.

[33] Markus Wenzel. Isar - a generic interpretative approach to readable formal proof documents. In

Yves Bertot, Gilles Dowek, André Hirschowitz, C. Paulin, and Laurent Théry, editors, *TPHOLs*, volume 1690 of *LNCS*, pages 167–184. Springer, 1999.